

# TLA+ Models of Parsl

## Project Report

TLA+ Parsl Audit Project

October 2026

### Abstract

This project develops a small executable TLA+ abstraction of Parsl for finding high-probability protocol errors. The primary focus is communication across process and thread boundaries; asynchronous interleavings and terminal-state handling are secondary. The result is a finite, repeatable audit connecting source transitions to TLC counterexamples, candidate safety protocols, and runtime probes against the installed Parsl implementation.

## 1 Research question

Distributed workflow systems can fail even when the user function is correct. A task can leave one queue and fail to arrive at the next component; a late retry result can overwrite a newer result; or a callback can mutate a registry while failure fan-out is traversing it. The central question is: *when a logical task crosses asynchronous boundaries, who owns it, and what must remain true when the next boundary fails?*

The model separates the following identities:

$$\text{logical task} \rightarrow \text{physical attempt} \rightarrow \text{transport message} \rightarrow \text{Future and monitoring state}$$

This separation lets the model classify an attempt-0 result as stale after attempt 1 has become current, rather than allowing it to resolve the logical Future a second time.

## 2 Model and evidence method

TLA+ describes states and transitions rather than implementation syntax. Each concrete source-backed finding follows a Current/Fixed discipline:

1. Current preserves the source-like transition and should produce a meaningful counterexample.
2. Fixed expresses a candidate safe protocol: retain ownership, isolate malformed input, check attempt generation, or snapshot a collection before callbacks.
3. A focused Python probe exercises the installed Parsl path when the boundary is executable.
4. The source location, trace, candidate fix, and limitations are recorded in the bug ledger.

The resulting evidence chain is:

$$\text{source transition} \rightarrow \text{Current TLC trace} \rightarrow \text{runtime probe} \rightarrow \text{Fixed invariant}$$

A passing Fixed run is evidence for finite constants and transitions. It is not an unbounded proof about Python, a real network, or every deployment.

### 3 Coverage

The model families cover:

- DAG dependencies, Futures, retries, cancellation, timeouts, and `join_app`;
- logical tasks and physical attempts, serializer identity, multipart frames, routes, acknowledgements, duplicates, and stale results;
- HTEX, Thread, Flux, MPI, Work Queue, TaskVine, and Radical-Pilot lifecycle paths;
- provider provisioning, scheduler status, cancellation, scale-in/scale-out, and cleanup;
- DataFuture readiness, partial files, retries, atomic publication, and transfer cleanup;
- monitoring queues, database bookkeeping, shutdown, persistence, heartbeat, and timeout behavior.

Table 1: Current project inventory

| Artifact                                       | Count |
|------------------------------------------------|-------|
| TLA+ modules                                   | 676   |
| TLC configuration files                        | 1,478 |
| Unittest methods                               | 761   |
| Runtime probe files                            | 490   |
| Foundational runtime entries                   | 486   |
| Foundational TLC configurations                | 754   |
| Bug ledger records                             | 318   |
| Unique numeric BUG identifiers (approximately) | 317   |

## 4 Representative findings

### 4.1 HTEX task dispatch send failure

In the HTEX interchange path, a pending task is removed from the queue before it is sent to a manager over ZeroMQ. If the send raises, the task is no longer pending, assigned, or terminalized. The Future can remain pending indefinitely. The Current model violates a no-task-loss invariant; the Fixed model retains ownership until successful transfer or emits an explicit terminal transport failure. This is recorded as BUG-335.

Recent communication refinements cover malformed command ingress (BUG-336), command-reply send failure (BUG-337), and heartbeat ACK send failure (BUG-338). A BUG-004 refinement covers synthetic manager-loss results that fail during transport. The shared safety principle is that a failed transport operation must preserve ownership for retry or make failure explicit and terminal.

### 4.2 Stale result after retry

If attempt 0 times out and attempt 1 succeeds, a delayed attempt-0 result must not overwrite the current Future. The model requires both logical task identity and the current physical-attempt generation. The stale result is consumed or rejected without changing terminal state.

### 4.3 Callback mutation during failure fan-out

Several failure paths iterate a live task dictionary while synchronously invoking `Future.set_exception`. A callback can remove an entry during iteration, causing an exception and leaving unrelated Futures pending. The candidate safe protocol snapshots entries before invoking callbacks.

## 5 Ledger distribution

The canonical ledger contains 318 records, including a refinement that reuses BUG-259, and about 317 unique numeric identifiers. The distribution below describes record assignment, not defect prevalence.

Table 2: Ledger record distribution

| Category                           | Records |
|------------------------------------|---------|
| Providers and scheduler adapters   | 92      |
| Executors and worker lifecycle     | 78      |
| Serialization and ZMQ transport    | 53      |
| Monitoring and database            | 32      |
| File staging and transfer          | 29      |
| Clock, heartbeat, and timeout      | 22      |
| join_app and memoization           | 9       |
| Core dataflow and Future lifecycle | 3       |

These records are source-backed protocol risks reproduced in the current implementation or in a tightly scoped test double. They are not automatically confirmed upstream defects.

## 6 Verification and reproduction

The latest complete regression passed all 754 foundational TLC configurations and all 486 runtime entries. The normal gate checks Fixed/safe configurations. A separate recent-model runner checks that intentional Current configurations still produce their expected counterexamples.

```
PYTHONPATH=/tmp/parsl-source \\  
PYTHON_BIN=/tmp/parsl-venv/bin/python \\  
PARSL_SOURCE=/tmp/parsl-source \\  
bash scripts/runtime_foundational_smoke.sh  
  
JAVA_BIN=/path/to/java \\  
TLA_JAR=/path/to/tla2tools.jar \\  
TLC_SIMULATE=100 \\  
bash scripts/tlc_foundational_smoke.sh
```

## 7 Interpretation and limits

The abstraction omits unbounded Python heaps, arbitrary object graphs, operating-system scheduling, real network timing, and many backend-specific implementation details. A Current counterexample demonstrates a reachable source-like behavior under the model; it does not imply that every production deployment fails. A runtime probe demonstrates behavior in a controlled setup; it is not a production-scale experiment. The ledger is a research record, not an upstream issue tracker.

The project stops expanding when the selected communication paths and asynchronous races have source-backed models, runtime evidence, documentation, and passing smoke suites. New model families require an explicit scope change.

## 8 Project artifacts

The repository contains the detailed [project report](#), [article and podcast plan](#), [bug ledger](#), and [coverage matrix](#). Component notes document the mapping from Parsl source paths to TLA+ modules and runtime probes.